

Improved Configuration Management

dm9051opts.h

```
// Add configuration validation
#ifdef DRV_INTR_MODE
#if !defined(ETHERNET_INTERRUPT_MODE) || (ETHERNET_INTERRUPT_MODE != 1)
#error "ETHERNET_INTERRUPT_MODE must be defined and set to 1 when using DRV_INTR_MODE"
#endif
#endif

// Add configuration structure
typedef struct {
    bool interrupt_enabled;
    bool force_10m;
    bool checksum_offload;
    uint8_t debug_level;
} dm9051_config_t;

// Add configuration validation function
static inline bool validate_config(const dm9051_config_t *config) {
    if (!config) return false;
    // Add validation logic
    return true;
}
```

Better Debug Support

dm_impl.c

```
// Enhanced debugging support
#if DM_ETH_DEBUG_MODE
#define DM9051_DEBUG_LOG(level, fmt, ...) do { \
    if (level <= current_debug_level) { \
        printf("[DM9051 %s] " fmt "\n", level_str_impl(level), ##__VA_ARGS__); \
    } \
} while(0)

void dump_registers_debug(void) {
    uint8_t regs[16];
    cspi_read_regs_info(regs);
    DM9051_DEBUG_LOG(DM9051_ETH_DEBUG_LEVEL_DEBUG,
        "Register dump: %02X %02X %02X %02X %02X %02X %02X %02X",
        regs[0], regs[1], regs[2], regs[3], regs[4], regs[5], regs[6], regs[7]);
}
#endif
```

please put dm9051_configure in cspi_core_reset

dm9051.checksum_offload

```
// Add to existing code or create new function
dm9051_status_t cspi_core_reset(const dm9051_config_t *config) {
    uint8_t reg_val;
    dm9051_status_t status = DM9051_OK;

    // Software reset
    cspi_write_reg(DM9051_NCR, NCR_RST);
    dm_delay_us(10); // Wait for reset to complete

    // Verify reset completed
    reg_val = cspi_read_reg(DM9051_NCR);
    if (reg_val & NCR_RST) {
        DM9051_LOG_ERROR("Reset failed to complete");
        return DM9051_ERR_RESET;
    }

    // Configure network control
    reg_val = 0;
    if (config->phy.full_duplex) {
        reg_val |= NCR_FDX;
    }
    cspi_write_reg(DM9051_NCR, reg_val);

    // Configure PHY mode
    if (config->phy.force_10m) {
        // Force 10Mbps mode
        uint16_t bmc_r = cspi_phy_read(PHY_BMCR);
        bmc_r &= ~(BMCR_SPEED_100 | BMCR_AUTONEG);
        cspi_phy_write(PHY_BMCR, bmc_r);
        DM9051_LOG_INFO("Forced 10Mbps mode");
    }

    // Configure checksum offload
    if (config->phy.checksum_offload) {
        cspi_write_reg(DM9051_TCSCR, TCSCR_IPCSE | TCSCR_TPCSE | TCSCR_UDPCSE);
        cspi_write_reg(DM9051_RCSCR, RCSCR_IPCE | RCSCR_TCPE | RCSCR_UDPE);
        DM9051_LOG_INFO("Checksum offload enabled");
    }

    // Configure interrupt if enabled
    if (config->interrupt.enabled) {
        uint8_t imr = IMR_PAR | IMR_PRM | IMR_PTM;
        if (config->interrupt.active_low) {
            imr |= IMR_LNKCHG;
        }
        cspi_write_reg(DM9051_IMR, imr);
        DM9051_LOG_INFO("Interrupts configured");
    } else {
        // Disable all interrupts
        cspi_write_reg(DM9051_IMR, 0x00);
    }

    // Configure receive control
    reg_val = RCR_DIS_LONG | RCR_DIS_CRC | RCR_RXEN;
    if (config->phy.promiscuous) {
        reg_val |= RCR_PRMSC;
    }
    cspi_write_reg(DM9051_RCR, reg_val);

    // Initialize statistics
    memset(&dm9051_stats, 0, sizeof(dm9051_stats_t));

    DM9051_LOG_INFO("Core reset and configuration complete");
    return status;
}
```

